



DESCRIPTIVE WRITTEN EXAM

COURSE NAME: Software Testing

COURSE CODE: CSA3711

Code of Conduct:

*I **PRADEEEP.S**(192521024) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:*Pradeep.s*

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1. Testing as an Engineering Activity – Banking Application

Introduction:

Testing should be treated as a systematic engineering activity rather than random checking. For the banking application, the team should follow a planned testing process.

1. Systematic Test Design:

- Study requirements and identify important banking functions.
- Prepare test scenarios and test cases before execution.
- Use equivalence partitioning, boundary value analysis and decision table testing.
- Give priority to critical functions such as login, money transfer and balance checking.

2. Tools and Standards:

- Use test management tools to maintain test cases and defects.
- Use automation tools for repeated regression tests.
- Follow coding, testing and documentation standards.
- Maintain clear defect reports with severity and priority.

3. Improvement in Defect Detection:

Planned testing gives better coverage and reduces missed defects. Automation also helps repeat important tests after every software change.

4. Techniques to Replace Ad-hoc Testing:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing
- State Transition Testing
- Regression Testing

Practical Example:

For a fund transfer of ₹1,000, test valid transfer, zero amount, negative amount, insufficient balance and maximum allowed amount. These systematic tests can find defects that random testing may miss.

Conclusion:

Engineering-based testing improves test coverage, defect detection, reliability and overall software quality.

2. Structured Testing Process for ATM Withdrawal**Introduction:**

An ATM requires a defined testing process because incorrect results can cause financial loss. Testing should be performed throughout the SDLC.

Stages:

1. Requirement Analysis – Identify withdrawal rules, PIN validation, balance rules and transaction limits.
2. Test Planning – Define scope, resources, test schedule and risks.
3. Test Case Design – Prepare valid, invalid and boundary test cases.
4. Test Environment Setup – Prepare ATM software, database, cards and test accounts.
5. Test Execution – Execute test cases and record actual results.
6. Defect Reporting – Report mismatches with severity, steps and expected/actual results.
7. Retesting – Test the corrected defect.
8. Regression Testing – Ensure that corrections have not affected other functions.
9. Test Closure – Prepare final test summary and lessons learned.

ATM Example:

Test withdrawal of ₹5,000 with sufficient balance, insufficient balance, invalid PIN, daily-limit violation and invalid amount.

Benefits:

A structured process reduces missed scenarios, improves consistency and makes defects easier to track.

Conclusion:

Following the process across SDLC improves ATM reliability, security and transaction accuracy.

3. Error, Defect and Failure in a Web Application**Introduction:**

Error, defect and failure are related but different testing terms.

1. Error:

A human mistake made during requirements, design or coding.

Example: A developer writes an incorrect condition for login.

2. Defect:

A flaw in the software caused by an error.

Example: The login program accepts an incorrect password because of the wrong condition.

3. Failure:

The visible incorrect behavior when the defective software is executed.

Example: A user enters a wrong password and the system still allows login.

Relationship:

Human Error → Defect in Software → Failure during Execution.

Transaction Example:

If the developer misunderstands the transaction amount rule, the code may contain a defect. During execution, an incorrect transaction amount may be processed, producing a failure.

Effect of Confusion:

If these terms are confused, defect reports become unclear and developers may not understand the actual cause.

Improvement:

Defect reports should include title, steps to reproduce, expected result, actual result, environment, severity and priority.

Conclusion:

Clear terminology improves communication, tracking and defect resolution.

4. Defect Clustering and Test Prioritization

Introduction:

Defect clustering means that a small number of modules usually contain a large number of defects. This principle helps testers focus their effort.

Application:

1. Collect defect history for all modules.
2. Identify modules with high defect counts.
3. Analyze common defect patterns.
4. Give higher testing priority to risky modules.
5. Perform regression testing on affected areas.

Other Useful Principles:

- Exhaustive testing is not possible, so testing must be prioritized.
- Testing should start early.
- Repeated tests may become less effective, so test cases should be reviewed and updated.
- Testing shows the presence of defects, not their complete absence.

Example:

In an e-commerce application, payment and checkout modules may contain more defects than the home page. Therefore, payment and checkout should receive more test cases and regression testing.

Conclusion:

Defect clustering helps use limited testing time efficiently and improves defect detection in high-risk areas.

5. Defect Classification and Defect Repository

Introduction:

A defect repository stores information about defects found during software development and testing. It can prevent repeated defects.

Defect Origin:

Defects may originate from incorrect requirements, design mistakes, coding errors, integration problems or configuration changes.

Classification:

- Requirement defects
- Design defects
- Coding defects
- Interface defects
- Data defects
- Performance defects
- Security defects

Using Repository Data:

1. Record each defect with module, cause, severity and status.
2. Search old defects before designing new tests.
3. Identify repeated defect patterns.

4. Add regression test cases for recurring problems.
5. Perform root-cause analysis.

Example:

If an incorrect tax calculation occurred in a previous release, the repository can be used to create regression cases for the same calculation in future releases.

Conclusion:

A well-maintained defect repository supports better test design, prevention and continuous quality improvement.

6. Mobile Banking Fund Transfer Crashes

Introduction:

Frequent crashes during fund transfer indicate that the application needs systematic testing and root-cause analysis.

Approach:

1. Reproduce the crash using the same steps.
2. Collect logs, device information and transaction details.
3. Check input validation, network handling and database operations.
4. Test valid, invalid and boundary transaction values.
5. Test network interruption during transfer.
6. Test insufficient balance and duplicate transfer attempts.
7. Report the defect clearly.
8. Retest after fixing and perform regression testing.

Example Test Cases:

- Valid transfer with sufficient balance.

- Transfer with insufficient balance.
- Transfer when network is disconnected.
- Very large transaction amount.
- Repeated tapping of Transfer button.
- Session timeout during transfer.

Improvements:

Use automated regression tests, crash monitoring, API testing and risk-based testing.

Conclusion:

Systematic testing and root-cause analysis reduce crashes and improve reliability of mobile banking transactions.

7. Testing and Debugging

Introduction:

Testing and debugging are related but different activities.

Testing:

Testing is the process of executing and evaluating software to find defects and verify requirements.

Debugging:

Debugging is the process of locating the cause of a defect and correcting the code.

Differences:

- Testing mainly identifies failures/defects.
- Debugging investigates the cause and fixes the defect.
- Testers commonly perform testing; developers commonly perform debugging, although responsibilities can vary.

Example:

If a fund transfer produces the wrong balance, testing detects the failure. Debugging traces the calculation and identifies the incorrect statement.

Problems with Confusion:

If testing is treated as debugging, testers may directly modify code instead of reporting evidence. This can reduce independent defect detection.

Structured Approach:

Testing → Defect Report → Developer Debugging → Fix → Retesting → Regression Testing.

Conclusion:

Separating testing and debugging gives clear responsibilities and improves software quality.

8. Cross-Browser Testing for E-Commerce

Introduction:

Different browsers may render and execute a web application differently. Therefore, cross-browser testing is important.

Strategy:

1. Identify supported browsers and versions.
2. Test important user journeys such as login, search, cart and payment.
3. Test different screen sizes and operating systems.
4. Compare expected and actual behavior.
5. Automate repeated browser tests where possible.
6. Record browser-specific defects.

Defect Causes:

- Different browser rendering engines.
- Unsupported CSS or JavaScript features.

- Browser-specific behavior.
- Responsive design problems.
- Incorrect third-party library support.

Techniques:

- Functional testing
- Compatibility testing
- Responsive testing
- Regression testing
- Automated browser testing

Tools:

Browser automation tools such as Selenium or Playwright can help execute tests across supported browsers.

Conclusion:

Systematic cross-browser testing increases compatibility, reliability and customer satisfaction.

9. Logical Errors in Payroll System

Introduction:

A logical error occurs when the program runs but produces an incorrect result because the implemented logic is wrong.

Classification:

The salary calculation defect can be classified as a coding/logic defect. It may affect basic salary, allowances, deductions, overtime or tax calculation.

Example:

Expected salary = Basic Salary + Allowances – Deductions.

If the program incorrectly subtracts an allowance, the output becomes wrong even though the program executes successfully.

Importance of Classification:

- Helps identify the source of defects.
- Supports root-cause analysis.
- Helps select suitable test cases.
- Helps identify modules needing more regression testing.

Prevention:

- Review requirements and formulas.
- Perform code reviews.
- Use unit testing for salary components.
- Use boundary and equivalence tests.
- Maintain regression test cases.
- Compare results with manually verified calculations.

Conclusion:

Correct defect classification helps improve test design and prevents logical calculation errors in future releases.

10. Structured Test Case Design

Introduction:

Random testing alone cannot provide systematic coverage. Test cases should be designed from requirements and risk.

Steps:

1. Understand requirements.

2. Identify test conditions.
3. Select suitable test design techniques.
4. Write test cases with inputs, steps, expected results and priority.
5. Review test cases.
6. Execute and record results.
7. Update tests based on defects.

Techniques:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Tables
- State Transition Testing
- Use-case testing
- Risk-based testing

Example:

For a password field accepting 8–20 characters, test below minimum, minimum, normal, maximum and above maximum values.

Benefits:

Structured design increases coverage, reduces duplication and improves defect detection.

Conclusion:

Replacing random testing with systematic test design makes testing more effective and measurable.

11. Data Inconsistency in Healthcare Application

Introduction:

Healthcare applications require accurate and consistent patient data across modules.

Possible Defect Origins:

- Incorrect database mapping
- Incorrect update logic
- Interface defects
- Data conversion errors
- Synchronization problems
- Incorrect requirements

Testing Approach:

1. Identify common patient data fields.
2. Compare data between modules.
3. Test create, update and delete operations.
4. Verify database values.
5. Test concurrent updates.
6. Perform integration and regression testing.

Example:

If a patient's address is updated in registration but not reflected in billing, an integration or data synchronization defect may exist.

Defect Tracking:

Reports should contain patient/test record ID, module, steps, expected result, actual result, severity and evidence.

Conclusion:

Systematic integration testing and clear defect tracking help maintain data integrity and reliability.

12. Repeated Defects and Defect Repository

Introduction:

Repeated defects indicate that previous defect information is not being effectively reused.

Approach:

1. Maintain a centralized defect repository.
2. Record defect description, module, cause, severity, status and fix.
3. Analyze repeated defects.
4. Perform root-cause analysis.
5. Convert important historical defects into regression test cases.
6. Review old defects during new test planning.

Identifying Patterns:

Defects can be grouped by module, type, severity, release and root cause. If the same module repeatedly produces defects, additional testing should be planned for it.

Example:

If login defects occur in three releases, testers should review the common cause and create stronger login validation and regression tests.

Conclusion:

Using repository data helps prevent recurring defects and improves future test planning.

13. Peak Usage Failure in Learning System

Introduction:

The failure during peak usage indicates that important performance scenarios were not included in testing.

Testing Gaps:

- Insufficient load testing

- No stress testing
- Inadequate concurrency testing
- Missing peak-user scenarios
- Limited performance monitoring

Test Planning:

Test planning should define expected users, response time, workload, risks and performance criteria.

Coverage:

Functional coverage alone is not enough. Performance scenarios must also be included.

Example:

If 5,000 students access an online exam at the same time, the system should be tested with a similar or higher simulated load before deployment.

Improvements:

- Load testing
- Stress testing
- Performance monitoring
- Capacity testing
- Automated performance tests
- Testing peak and abnormal conditions

Conclusion:

Including realistic performance scenarios in test planning improves reliability during high usage.

14. Early Involvement of Testers in SDLC

Introduction:

Testers should be involved from the early stages of the SDLC instead of joining only after coding.

Role Across SDLC:

- Requirements: identify ambiguity and missing conditions.
- Design: review testability and risks.
- Development: discuss unit and integration test needs.
- Testing: design and execute test cases.
- Release: verify quality and risks.
- Maintenance: support regression testing.

Benefits:

Early involvement helps detect requirement defects before they become expensive coding defects.

Example:

If a requirement says a transaction has a maximum limit but does not define the exact limit, the tester can identify this ambiguity before development.

Team Collaboration:

Use requirement reviews, design reviews, shared defect tracking and regular communication between developers and testers.

Conclusion:

Early tester involvement reduces rework and improves overall product quality.

15. Incomplete Test Coverage

Introduction:

A system may work correctly for tested cases but fail in untested conditions. This happens when test coverage is incomplete.

Identifying Gaps:

- Review requirements against test cases.

- Identify untested conditions and paths.
- Check boundary and negative cases.
- Analyze defect history.
- Use coverage measures.

Example:

For a login function, testing only correct username and password is incomplete. Invalid username, invalid password, empty fields, locked account and session timeout should also be tested.

Improving Test Design:

Use equivalence partitioning, boundary value analysis, decision tables, state transition testing and risk-based testing.

Validation:

Test expected results, error messages, security conditions and integration behavior.

Conclusion:

Better coverage increases confidence that important system behaviors have been tested.

16. Improving Defect Reporting in Logistics Application

Introduction:

Poor defect reports delay defect identification and resolution. Clear reporting improves communication between testers and developers.

A Good Defect Report Contains:

- Defect ID and title
- Module
- Environment
- Preconditions

- Steps to reproduce
- Expected result
- Actual result
- Severity
- Priority
- Evidence such as screenshots or logs

Example:

Title: Delivery status not updated after shipment confirmation.

Expected: Status changes to Shipped.

Actual: Status remains Processing.

Benefits:

Structured reports reduce misunderstanding, speed up debugging and make defect tracking easier.

Reporting Standards:

Use a common defect template, clear terminology, consistent severity definitions and proper status values.

Conclusion:

Standardized defect reporting improves communication and reduces resolution time.

17. Process-Based Testing for Consistency

Introduction:

A process-based testing approach uses standardized activities, templates and rules across modules.

Approach:

1. Define a common testing process.
2. Use standard test case templates.

3. Define severity and priority levels.
4. Use common entry and exit criteria.
5. Maintain a central defect repository.
6. Conduct test reviews.
7. Measure test coverage and defect trends.

Example:

For login, payment and reporting modules, all testers should follow the same process: requirement review → test design → execution → defect reporting → retesting → regression.

Benefits:

- Consistent quality
- Better documentation
- Easier tracking
- Reduced missed tests
- Comparable results across modules

Conclusion:

Standardized processes make testing repeatable, measurable and reliable.

18. Edge Cases and Boundary Testing

Introduction:

Edge cases are unusual or extreme conditions that may be missed during normal testing. Boundary testing is useful for finding defects near limits.

Why Missed:

Testers may focus only on normal values and ignore minimum, maximum, empty, zero or unusually large inputs.

Boundary Testing:

For an input range of 1 to 100, test values such as 0, 1, 2, 99, 100 and 101.

Steps:

1. Identify input limits.
2. Test just below the minimum.
3. Test minimum and just above minimum.
4. Test just below maximum.
5. Test maximum and just above maximum.
6. Record expected and actual results.

Benefits:

Boundary testing finds incorrect comparison operators, validation errors and limit-handling defects.

Conclusion:

Including edge and boundary cases improves reliability and reduces post-deployment failures.

19. Defect Documentation and Repository**Introduction:**

Without proper defect documentation, organizations cannot easily track, analyze or prevent problems.

Repository Information:

A defect repository should contain ID, title, description, module, environment, steps, expected result, actual result, severity, priority, owner, status, root cause and resolution.

Benefits:

- Easy defect tracking
- Historical analysis
- Identification of repeated defects

- Better communication
- Support for regression testing

Example:

A database defect that caused incorrect customer information can be recorded with its root cause and fix. The same information can later be used to create regression tests.

Improvements:

Use a centralized repository, mandatory fields, standard terminology, regular reviews and links between defects and test cases.

Conclusion:

Good documentation converts defect information into useful knowledge for future testing and prevention.

20. Redesigning Testing for Banking Transaction Failures

Introduction:

Repeated transaction failures show that the current testing process is not sufficiently systematic.

Redesigned Approach:

1. Analyze requirements and transaction rules.
2. Identify high-risk functions.
3. Prepare test scenarios and test cases.
4. Apply equivalence partitioning and boundary testing.
5. Perform functional and integration testing.
6. Test invalid transactions and network failures.
7. Automate regression tests.
8. Record and classify defects.

9. Retest fixes and perform regression testing.

10. Review defect trends after each release.

Example Test Cases:

- Valid transaction with sufficient balance.
- Insufficient balance.
- Invalid account number.
- Maximum transaction limit.
- Network interruption.
- Duplicate transaction request.
- Session timeout.

Defect Detection:

Risk-based testing focuses more effort on critical transaction modules. Historical defect data can also guide test priorities.

Conclusion:

A structured, risk-based and regression-focused approach improves transaction reliability, detects defects earlier and increases customer confidence.